Closing Manager: dependency review and the path to running it ourselves

The month-end close macro works, but it still reaches two Capgemini-hosted systems. This briefing sets out what it depends on, what that risks, and what we need to take over.

FILE REVIEWED
Closing_Manager_IP_Legacy_changed_v3.xlsm

METHOD
Static source review — no macro executed

SCOPE
26 VBA modules · ~5,800 lines

THE SHORT ANSWER

Almost — and the gap is smaller than it looks. The macro's real work already runs entirely on the user's PC. What remains outside our control is **configuration, a log, and one small executable**, all hosted by Capgemini. Nothing there requires their infrastructure.

2
Capgemini-hosted systems still required at run time

4
settings tables held on their SharePoint, not in our file

0
of those dependencies are technically hard to replace

1 What it does

An Excel VBA macro that runs the month-end close end to end:

- **Drives SAP** via SAP GUI Scripting — runs the closing transactions (SE16 table extracts, `ZGLRME`, `ZGE132`, `AA02`, `ZGLGWUL` and others), exports the data to flat files, reads it back into the workbook and posts the closing entries.
- **Creates the PDFs** — sets SAP's front-end printer to `PDFCreator` and prints each report out to `\pdf\final\`.
- **Merges the PDFs** — calls `GiosPSMC.exe` to concatenate the report set into a single pack, then files it as `<CC> Closure <MM> <YYYY>.pdf` under `C:\_Files to Transfer\MONTH END CLOSE\`.
- **Reads and writes configuration** over SOAP to a Capgemini SharePoint site — the subject of this review.

One design point that drives most of the fragility: SAP is driven **by screen element ID**, not through a BAPI or other interface. The macro is a screen-scraper, so it is sensitive to SAP UI changes, unexpected pop-ups and authorisation differences.

2 What it depends on

Five dependencies. The marker shows who controls each one.

Capgemini SharePoint NOT OURS

Holds the macro's **settings tables** and receives a **log of every close we run**. Read at startup over SOAP (`Lists.asmx`), written to during runs. Authenticates as the logged-in Windows user; requires the corporate network or VPN.

```
https://troom-x.capgemini.com/sites/InternationalPaper/r2a/_vti_bin/Lists.asmx
```

Risk: if the site is retired or we are off their network, the macro loads **no settings and reports no error** — it carries on regardless. It also means our closing activity is recorded on a vendor system.

Capgemini Poland file server NOT OURS

A share at Capgemini's Kraków site, holding the PDF merge utility `GiosPSMC.exe`. The macro copies it locally on first run.

```
\\pl-krabpo-fsc01\ipa$\R2R\R2R - IP EU GL West\USEFUL\pdf\merger\GiosPSMC.exe
```

Risk: low and easily removed — it is a one-time file copy. Once the utility is on the PC the share is never touched again.

SAP OURS — KEEP

Must be open and logged in on the same PC, with GUI scripting enabled both client-side and on the server (`sapgui/user_scripting`). The user needs authorisation for the driven transactions.

Legitimate dependency — SAP is the source of the numbers. This one stays.

PDFCreator OURS TO DEPLOY

Third-party desktop software. The Windows printer must be named exactly `PDFCreator`, and its auto-save must write to the folder the macro polls.

Under our control — a standard desktop install, no external server involved. Version-sensitive, so worth pinning a known-good build.

Local folders OURS

Working folders under `C:\pdf\`, the delivered pack under `C:\_Files to Transfer\MONTH END CLOSE\`, plus temporary SAP exports written beside the workbook itself.

Under our control — but the workbook must sit in a **genuine local folder**. Run from OneDrive or SharePoint it fails, because its own path resolves to a URL (see section 4).

3 What is actually held on their SharePoint

This is the crux of the independence question. It is not accounting data — it is **configuration plus an activity log**.

LIST	CONTENTS	USE
CCCrossList cost-centre settings	Per cost centre: posting block, location-closed flag, CPC allowed/not-allowed under GAAP, comments	Read at startup into the `config` sheet; also maintained from a form inside the workbook
ClosingVariants SAP report settings	SAP variant name, currency and FC/USD flag per cost centre	Read at startup
ProfitCenters reference data	Profit-centre reference list	Read at startup; cleared and rewritten by the macro
ClosingTracker activity log	User name, cost centre, period, year, timestamp, success flag	Written during runs; also read to confirm data was refreshed for the period

All four are **small settings tables** that would sit comfortably in a worksheet or on our own SharePoint. There is no technical reason for them to live on Capgemini infrastructure.

Two points to raise with Capgemini

1. Our close activity is logged to their system. Every run writes the Windows user name, cost centre, period and timestamp into `ClosingTracker` on their SharePoint. We should confirm this is agreed, who can access it, and what the retention is.

2. Failure is silent. The SOAP calls have no status or error checking. If the endpoint is unreachable the macro simply loads nothing and continues — closing on empty or stale configuration rather than stopping. This should be fixed regardless of where the data ends up living.

Encouraging precedent: this "v3" build had already begun decoupling. Several SharePoint calls are commented out in the code — including the lookup that fetched the user's SAP ID, now replaced by simply prompting the user. `Closing.bas` no longer makes any SharePoint calls at all. The work was started; it just was not finished.

4 What stops a run today

ISSUE	EFFECT	STATUS
Working-drive mismatch coding defect	On a PC with a D: drive, the folders and merge utility are created on C: but read from D:. The close appears to work right up to printing, which then silently produces nothing.	FIXED IN V4
Run from OneDrive / SharePoint environment	The macro writes SAP exports beside the workbook. From a cloud location `ThisWorkbook.Path` returns a URL, which the SAP download dialog and file I/O cannot use, so the run fails at the first export.	GUARDED IN V4
SAP absent or not scriptable precondition	Reports "You are not logged in SAP." Also binds the first open session only, so it can drive the wrong window if several are open.	PARTLY HANDLED
SAP screen changes design fragility	Hundreds of hard-coded screen element IDs with little error handling. An SAP upgrade, changed variant or unexpected pop-up halts the macro mid-run.	ONGOING RISK
PDFCreator missing dependency	Previously launched an unchecked path and crashed; now reports clearly.	FIXED IN V4
Off network / no VPN external dependency	Settings silently fail to load; the merge utility cannot be fetched if not already local.	NEEDS DECISION

5 Preflight Check — can users run it on its own?

Yes. It runs standalone, and it changes nothing.

`PreflightCheck` **is a self-contained macro** added in V4 specifically so users can verify their environment **before** committing to a close. It does not start the close, does not touch SAP beyond detecting whether a session exists, creates no folders, and writes no files. It is safe to run at any time, as often as needed.

How to run it: press **Alt + F8**, select **PreflightCheck**, click **Run**. It appears in the macro list automatically. For everyday use, add a button on the `START` sheet (*Developer → Insert → Button*) and assign it to the same macro.

It reports on all six preconditions in one dialog:

```
CLOSING MANAGER  -  PREFLIGHT CHECK
------------------------------------
[OK] Workbook location is local.
[OK] Working drive C:\ is present.
[X]  SAP GUI not logged in, or GUI scripting is disabled.
[OK] PDFCreator is installed.
[~]  PDF merger will be copied from the network share on first run.
[OK] Cost Centre = 1234.
------------------------------------
NOT READY - resolve the [X] items above.
```

This turns every dependency in section 2 into a plain pass/fail before anyone starts a period close — including whether the network-hosted merge utility is still needed on that machine.

6 Becoming self-sufficient

Each step stands alone and can be done independently, in roughly this order.

1 **Take a permanent copy of the merge utility**

Copy `GiosPSMC.exe` from the Kraków share to a location we own, or deploy it alongside the workbook. This removes that dependency outright and is the quickest win. Confirm with Capgemini what the utility is and how it is licensed before we redistribute it.

2 **Request an export of the four settings tables**

Ask for current contents of `CCCrossList`, `ClosingVariants`, `ProfitCenters` and `ClosingTracker`, plus who maintains them today and how often they change. This defines exactly what we are taking over.

3 **Decide where the settings should live**

Either hold them in hidden worksheets in the workbook — simplest, and removes the server entirely — or move them to a list on our own SharePoint. The choice turns on how many people maintain them and whether we need an audit trail.

4 **Repoint or disable the SharePoint calls**

V4 already isolates the address in a single named constant (`CM_SP_BASE`), so redirecting to our own site is a one-line change rather than a hunt through seven call sites. Disabling the logging entirely is equally straightforward.

5 **Make failure loud instead of silent**

Add a check that the configuration actually loaded, so an unreachable endpoint stops the run with a clear message instead of closing on empty settings. **This is the most important safety item on the list** and is worth doing before, or independently of, any migration.

6 **Settle ownership and support of the macro**

The code header names an owner and reviewer on a Capgemini automation register. If this automation is becoming ours, we need the source, the right to modify it, and a named internal owner.

> **Already delivered.** A hardened **V4-CIO** build of the workbook is available now. It fixes the working-drive defect, blocks running from OneDrive, prevents the macro hanging indefinitely when a step stalls, fails clearly when the merge utility or PDFCreator is missing, and adds the **Preflight Check** above. It also gathers both external addresses into named constants — which is what turns steps 1 and 4 into configuration changes rather than a code rewrite.

Where this leaves us

Nothing in this automation genuinely requires Capgemini's servers. What sits there is **configuration we should own in any case** and **a log of our own activity**. The practical path is to take the merge utility, take the settings, point the macro at a home we control, and make it fail loudly when something is missing. After that the only external system left is SAP — which is exactly as it should be.

Prepared by the Continuous Improvement Team from a full static review of the workbook's VBA source. No macro was executed and no live system was contacted. Addresses and list names shown are those written into the workbook's own code. Contains no credentials or accounting data.